SlabMaster Automated Test Suite Summary & Specification

Executive Overview

SlabMaster utilizes a high-performance automated testing architecture powered by **Vitest 2.1.9** and **jsdom**, configured with native TypeScript support and zero runtime overhead.

The test suite consists of **28 automated tests across 7 specialized test suites**, achieving 100% test passing across all core modules and newly implemented Moraware parity features.

Testing Architecture & CI/CD Gate

- **Engine:** Vitest 2.1.9 + `@testing-library/react` + `jsdom`.
- **Performance:** Executes all 28 tests in **~1.8 seconds**.
- **Azure CI/CD Quality Gate:** Automated test execution is embedded directly into the production build pipeline (`vitest run && tsc && vite build`). Any test failure automatically prevents deployment to Azure Static Web Apps.
- **Developer Experience:** Supports headless execution, real-time terminal watch mode, and an interactive browser dashboard via `@vitest/ui`.

Complete Test Suite Catalog (28 Tests Total)

```
Test Files  7 passed (7)
     Tests  28 passed (28)
```

#	Test Suite File	Domain / Module	Tests Passed	Key Capabilities Tested
1	`slabInventory.test.ts`	Slab Inventory & Remnants	**5**	Dimensional SQFT math, remnant linkage, plant scoping, status filtering, barcode encoding
2	`purchasing.test.ts`	Purchasing & POs	**4**	PO line item valuation, cumulative SQFT ordered, partial receiving, full dock receiving
3	`shopFloor.test.ts`	Shop Floor Kiosk	**4**	Station queue filtering, one-tap stage transitions, rush prioritization, machine workload
4	`formsTableMatrixRunner.test.ts`	Table Matrix & SAP Sheets	**4**	SAP Config Sheet schema, matrix column IDs, CTOP & splash summation, default rows
5	`customAttributesUtils.test.ts`	Custom Attributes Engine	**4**	Multi-type validation, currency formatting, select option checks, regional plant scoping
6	`tableMatrixUtils.test.ts`	Matrix Grid Utilities	**3**	Dynamic column summation, string/number sanitization, row preset initialization
7	`helpDocAnchors.test.ts`	Documentation (Rule 6)	**4**	Static serving check, 11 deep anchor targets, nav link verification, versioning display

---

Detailed Test Breakdown by Suite

1. Slab Inventory & Remnant Management (`slabInventory.test.ts`)

Focus: Dimensional precision, remnant lineage tracking, multi-plant isolation, and thermal shop labeling.

1. **Dimensional SQFT Calculation:**
   - **What is tested:** Verifies formula `SQFT = (Length × Width) / 144`, rounded to two decimal places.
   - **Test cases:** Evaluates standard slab `126" × 63"` (yields `55.13` SF) and jumbo slab `130" × 65"` (yields `58.68` SF).
   - **Boundary handling:** Validates that non-positive inputs (0 or negative dimensions) return `0` without throwing NaN or runtime errors.
2. **Remnant Offcut Lineage Linkage:**

- **What is tested:** When a remnant is created from a parent slab, verifies that `isRemnant` is set to `true`, the `parentSlabSerial` accurately points to the parent slab's serial number, and a unique `-R1` serialized identifier is assigned.
- **Assertions:** Validates material name, thickness (e.g. `3CM`), physical rack bin location (e.g. `Remnant Rack R-02`), and recalculated remnant SQFT (`48" × 30" = 10.0 SF`).

3. **Multi-Facility Plant Scoping:**
   - **What is tested:** Verifies that slab queries can be strictly partitioned by regional facility code (`ATL`, `PHX`, `TUC`) or aggregated globally across `ALL`.
   - **Assertions:** Slabs allocated to Atlanta only surface under Atlanta queries, isolating shop inventories across national plants.
4. **Lifecycle Status & Search Query Filtering:**
   - **What is tested:** Tests compound filtering combining lifecycle states (`Available`, `Allocated`, `Consigned`, `Consumed`) with freeform text searches.
   - **Assertions:** Verifies searches against material species (`Cambria`, `Silestone`), serial numbers, and physical rack locations (`A-Frame`).
5. **Thermal Barcode Label Generation:**
   - **What is tested:** Validates formatting of thermal barcode strings for thermal label printers.
   - **Payload format:** `SLAB:{serial}|MAT:{material}|DIM:{LxW}|SQFT:{sqft}|RACK:{rack}`.

---

2. Purchasing & Purchase Orders (`purchasing.test.ts`)

Focus: Purchase order financial valuation, material delivery tracking, and direct dock-receiving into inventory.

1. **PO Financial Valuation Calculation:**
   - **What is tested:** Multiplies unit quantities by individual unit costs and calculates the accurate aggregate purchase order total (e.g. 4 slabs @ $1,250 + 2 slabs @ $1,600 = $8,200.00).
2. **Cumulative SQFT Ordered Aggregation:**
   - **What is tested:** Computes the total square footage of stone ordered across all disparate line items (e.g. `4 × 55 SF + 2 × 48 SF = 316 SF`).
3. **Partial Dock Receiving Workflow:**
   - **What is tested:** When receiving a partial quantity (e.g., 2 out of 4 slabs), the PO status automatically shifts to `Partial`, quantity received increments to `2`, and 2 new serialized slabs with designated rack staging locations are generated.
4. **Full Fulfillment Lifecycle Transition:**

- **What is tested:** When all remaining line items are received at the loading dock, the PO status automatically transitions from `Partial` to `Received`, verifying full order completion.

---

3. Shop Floor Kiosk View (`shopFloor.test.ts`)

Focus: Machine station queues, one-tap job progression, rush job prioritization, and machine workload balancing.

1. **Station Cut Queue Filtering:**
   - **What is tested:** Filters shop jobs by their designated machine station: `Saw 1 (Bridge Saw)`, `Saw 2 (Waterjet CNC)`, `CNC Router`, or `Polish Line`.
   - **Assertions:** Ensures machine operators only see jobs staged for their specific workstation.
2. **One-Tap Stage Advancement:**
   - **What is tested:** Simulates machine operator tablet interactions advancing a job through stages:
     - `queued` ➔ `in_progress` ➔ `completed`.
   - **Boundary handling:** Advancing an already `completed` job remains idempotent (`completed`).
3. **Rush Job Prioritization:**
   - **What is tested:** Verifies that emergency/rush jobs (`isRush: true`) are automatically sorted to the absolute top of the cutting queue, regardless of numerical order date or standard priority scores.
4. **Active Machine SQFT Workload Balancing:**
   - **What is tested:** Aggregates total active square footage queued for each station, giving shop supervisors real-time visibility into machine bottlenecks and capacity utilization.

---

4. SAP Config Sheet & Table Matrix Integration (`formsTableMatrixRunner.test.ts`)

Focus: Multi-row, multi-column tabular sub-grids and Moraware SAP Configuration Sheet parity.

1. **SAP Configuration Sheet Template Verification:**
   - **What is tested:** Verifies that the default system template `ft_sap_config_sheet` exists with the title `SAP / Builder Room Takeoff & Config Sheet`.
2. **Table Matrix Column Schema Validation:**
   - **What is tested:** Validates that the matrix contains all mandatory configuration columns:
     - `room` (Room Name)
     - `ctop_type` (Countertop Type)
     - `material` (Species / Color)

- `ctop_sqft` (Countertop SQFT)
- `splash_sqft` (Backsplash SQFT)
- `sink_model` (Sink Specification)
- `edge_profile` (Edge Detail)

3. **Dynamic Multi-Room Column Summation:**
   - **What is tested:** Accurately sums decimal values across rows (Kitchen Perimeter, Island, Primary Bath) for both Countertop SQFT (`96.0 SF`) and Splash SQFT (`24.5 SF`).
4. **Default Matrix Row Initialization:**
   - **What is tested:** Confirms that new form instances automatically populate the 6 standard builder takeoff rooms (`Kitchen Perimeter` , `Kitchen Island` , `Primary Bath` , `Secondary Bath` , `Powder Room` , `Laundry`).

---

5. Custom Attributes Engine (`customAttributesUtils.test.ts`)

Focus: Runtime schema extension, multi-type data validation, and facility scoping.

1. **Required Attribute Enforcement:**
   - **What is tested:** Validates that fields marked `isRequired: true` reject empty strings or whitespace while accepting valid input.
2. **Currency Validation & Formatting:**
   - **What is tested:** Rejects non-numeric strings and formats raw numbers into standard accounting format (e.g. `2400` ➔ `"$2,400.00"`).
3. **Select Dropdown Choice Validation:**
   - **What is tested:** Verifies that selected values belong to the defined `options` array (e.g., `'Eased'` passes; unlisted `'Chiseled'` fails).
4. **Regional Plant Scoping:**
   - **What is tested:** Verifies that custom fields scoped to `ATL` or `PHX` are only surfaced when editing jobs in those respective facilities, while `ALL` fields appear globally.

---

6. Table Matrix Sub-Grid Utilities (`tableMatrixUtils.test.ts`)

Focus: Low-level string parsing, sanitization, and edge case resilience.

1. **Column Sum with Mixed Data Types:**

- **What is tested:** Successfully extracts and sums numbers from mixed inputs (numbers, strings with decimals, and currency symbols).

2. **Empty / Malformed Row Resilience:**
   - **What is tested:** Empty arrays or `null` / `undefined` row data gracefully return `0` rather than throwing uncaught exceptions.
3. **Preset Matrix Row Generation:**
   - **What is tested:** Transforms an array of room names into structured matrix rows initialized with default values.

---

7. Application Documentation & Anchor Compliance (`helpDocAnchors.test.ts`)

Focus: Enforcement of Application Development Guidelines Rule 6.

1. **Static Serving Check:**
   - **What is tested:** Confirms `frontend/public/help.html` exists on disk and is bundled into the distribution directory.
2. **Mandatory Anchor Target Presence:**
   - **What is tested:** Verifies that every application module has a matching anchor section:
     - `#overview`
     - `#accounts`
     - `#jobs`
     - `#calendar`
     - `#inventory`
     - `#purchasing`
     - `#shop-kiosk`
     - `#custom-attributes`
     - `#table-matrix`
     - `#forms`
     - `#settings`
3. **Navigation Anchor Links Verification:**
   - **What is tested:** Validates that the sticky sidebar navigation contains valid matching `href="#..."` links to each section.
4. **Copyright & Version Synchronization:**

- **What is tested:** Validates that the persistent copyright and version text (`© 2026 SlabMaster | v1.0.0`) is present in `help.html` .

How to Run & Watch Tests

1. One-Shot Test Run (Headless)

Run all 28 tests with a complete summary:

```
cd frontend
npm test
```

2. Interactive Terminal Watch Mode

Continuously monitors source files and re-runs tests instantly on save:

```
cd frontend
npm run test:watch
```

3. Interactive Visual Browser Dashboard

Opens the Vitest UI in your web browser with graphical hierarchy, execution timers, and single-click re-testing:

```
cd frontend
npm run test:ui
```

URL: `http://localhost:51204/__vitest__/`

4. Cloud Verification in GitHub Actions / Azure

Every git push to `main` executes `vitest run` as part of the production build step in Azure Static Web Apps:

- Live logs accessible at: **GitHub Actions Runs**
